

INFORME DE IMPLEMENTACIÓN

Sistema de Análisis de Rendimiento Académico

Estructura General

El sistema se desarrolló en C++ siguiendo un enfoque modular con separación de responsabilidades:

Módulo	Archivos	Función
Principal	main.cpp	Menú y control de flujo
Lectura	lectorArchivos.cpp/.h	Carga de archivos .lfp
Validación Léxica	validacionLexica.cpp/.h	Validación de formato de tokens
Validación Semántica	validaciones.cpp/.h	Validación de reglas de negocio
Reportes	reportes/*.cpp + headers	Generación de 5 reportes HTML
Estructuras	estudiante.h, curso.h, nota.h	Definición de datos

Flujo de Trabajo Implementado

1. Usuario elige opción en menú

2. Si es carga de archivo:

- Abrir archivo
- Leer línea por línea
- Separar campos por coma (split)
- Validación léxica (tipo de dato)
- Validación semántica (reglas)
- Guardar en vector si todo ok

3. Si es generación de reporte:

- Verificar archivos cargados
- Procesar datos según algoritmo
- Generar archivo HTML

4. Volver al menú o salir

Funciones Clave Implementadas

```
// Validación léxica
bool esIntLexico(string digito) ; // verifica que sea número entero
bool esStringLexico(string s) ; // verifica que sea solo letras
bool esDecimalLexico(string decimal); // verifica formato decimal
bool identificadorCiclo(string ciclo) ;// verifica formato "1S" o "2S"

// Validación semántica
bool carreraValidaUsac(string carrera); // compara con carreras válidas
bool validacionNota(string nota) ; // verifica rango 0-100
bool validacionCarnet(string carnet) ; // verifica longitud 9 dígitos

// Reportes
void generarReporteCursos(); // estadísticas por curso
void generarReportesRendimiento(); // rendimiento por estudiante
void generarReporteMejoresEstudiantes(); // top 10
void generarReportesMasReprobados(); // cursos con reprobación
void generarReportesAnalisisCarrera(); // análisis por carrera
```

RETOS TÉCNICOS ENCONTRADOS

Reto 1: Separación de campos por comas (split)

Problema: C++ no tiene una función split nativa como otros lenguajes.

Solución: Usar getline() con delimitador para leer campo por campo:

```
while (getline( [&] archivo, [&] carnet, delim:',')) {

    getline( [&] archivo, [&] nombre, delim:', ' );
    getline( [&] archivo, [&] apellido, delim:',');
    getline( [&] archivo, [&] carrera, delim:',');
    getline( [&] archivo, [&] semestre);
```

Reto 2: Espacios en blanco al final de líneas

Problema: Los archivos podían tener espacios, tabs o saltos de línea que afectaban las validaciones.

Solución: Crear función eliminarEspacio() que limpia caracteres no deseados:

```

string eliminarEspacio(string espacio) {
    string concatenacion;
    for (char es: espacio) {
        if (es != '\n' && es != '\r' && es != '\t' && es != ' ') {concatenacion += es; }
    }
    return concatenacion;
}

```

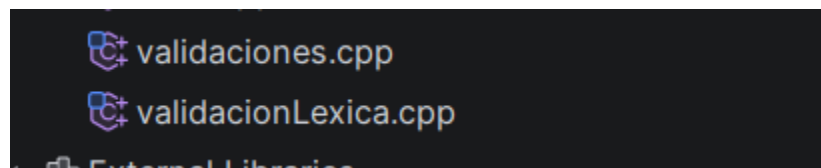
Reto 3: Distinguir entre validación léxica y semántica

Problema: Al principio se mezclaban ambos tipos de validación.

Solución: Separar en dos archivos distintos:

validacionLexica.cpp: solo verifica formato (es número?, es texto?)

validaciones.cpp: verifica reglas de negocio (carreras válidas, rangos)



Reto 4: Ordenamiento para top 10 y reprobación

Problema: Había que ordenar estructuras por un campo específico.

Solución: Usar método burbuja simple y efectivo:

```

/*Ordenamiento de Burbuja */
for (int l=0; l<listasPromedios.size()-1; l++) {
    for (int h=0; h< listasPromedios.size()-l-1; h++) {
        if (listasPromedios.at(h).promedio < listasPromedios.at(h+1).promedio) {
            swap(a [&]listasPromedios.at(h), b: [&]listasPromedios.at(h+1));
        }
    }
}

```

Reto 5: Rutas de archivos relativas

Problema: Al ejecutar desde CLion, la ruta "../data/" no siempre funcionaba igual.

Solución: Dejar ruta fija en el código y documentar que los archivos deben estar en carpeta data

```
if (opc == 1) {
    string nameArch = "../data/estudiantes.lfp";
```

SOLUCIONES APLICADAS

Problema	Solución Implementada
No hay split nativo	getline con delimitador coma
Espacios en líneas	función eliminarEspacio()
Validaciones mezcladas	separación léxica/semántica
Ordenamiento	método burbuja
HTML mal formado	plantilla fija con apertura/cierre
Rutas variables	ruta fija "../data/"

CAPTURAS DEL MENÚ EN EJECUCIÓN

Menú Principal

```
Run Practica_Unica x
" C:\Users\josev\OneDrive\Documentos\Documentacion Semest
=====
SISTEMA DE ANALISIS ACADEMICO
=====
1. Cargar Archivos Estudiantes
2. Cargar Archivos Cursos
3. Cargar Archivos Notas
4. Generar Reporte: Estadísticas por Curso
5. Generar Reporte: Rendimiento por Estudiante
6. Generar Reporte: Top 10 Mejores Estudiantes
7. Generar Reporte: Cursos con Mayor Reprobacion
8. Generar Reporte: Analisis por Carrera
9. Salir
Seleccione una opcion:
```

Carga de Estudiantes Exitosa

```

-----
Procesando Estudiante
-----
[LEXICO] OK
[SEMANTICO] OK
>>> Estudiante agregado Exitosamente
-----
Procesando Estudiante
-----
[LEXICO] OK
[SEMANTICO] OK
>>> Estudiante agregado Exitosamente
-----
Procesando Estudiante

```

```

=====
MOSTRAR ESTUDIANTES
=====
Carnet:202012345
Nombre: Juan
Apellido: Perez
Carrera: civil
Semestre: 5
-----
Carnet:202013456
Nombre: Juan

```

Intento de Reporte Sin Archivos

```

Seleccione una opcion:
4

No se Encuentran Cursos en la Lista=====

```

Generación de Reporte Exitosa

```

Seleccione una opcion:
6

Top Mejores Estudiantes
Generacion HTML Mejores Estudiantes Ok
=====

```

RESULTADOS DE REPORTES GENERADOS

Reporte: estadisticasCursos.html

```
<head>
<title>Reportes Cursos </title>
</head>
<hr>
<body>
<p>Nombre del Curso: Lenguajes Formales y de Programacion</p>
<p>Cantidad Estudiantes: 3</p>
<p>Promedio: 87.8333</p>
<p>Nota Maxima: 100</p>
<p>Nota Minima: 78</p>
<p>Mediana: 85.5</p>
<p>Desviacion Estandar: ... 9.13175</p>
<hr>
<body>
<p>Nombre del Curso: Inteligencia Artificial 1</p>
<p>Cantidad Estudiantes: 2</p>
<p>Promedio: 96</p>
<p>Nota Maxima: 100</p>
<p>Nota Minima: 92</p>
<p>Mediana: 96</p>
<p>Desviacion Estandar: ... 4</p>
<hr>
```

Reporte: mejoresEstudiantes.html

```
7 <body>
8 <h1>🏆 Top 10 Mejores Estudiantes</h1>
9 <hr>
0 <p><strong>Posición:</strong> 1 🏆</p>
1 <p><strong>Nombre:</strong> Jose</p>
2 <p><strong>Carnet:</strong> 202407095</p>
3 <p><strong>Carrera:</strong> Sistemas</p>
4 <p><strong>Semestre:</strong> 5</p>
5 <p><strong>Promedio:</strong> 100</p>
6 <hr>
7 <p><strong>Posición:</strong> 2 🏆</p>
8 <p><strong>Nombre:</strong> Juan</p>
9 <p><strong>Carnet:</strong> 202012345</p>
0 <p><strong>Carrera:</strong> civil</p>
1 <p><strong>Semestre:</strong> 5</p>
2 <p><strong>Promedio:</strong> 88.75</p>
3 <hr>
4 <p><strong>Posición:</strong> 3 🏆</p>
```

Reporte: CursosMasReprobados.html

```

<body>
<h1> Cursos Mas Reprobados </h1>
<hr>
<p><strong>codigo del Curso:</strong> 412</p>
<p><strong>Nombre del Curso:</strong> Estadística 1</p>
<p><strong>Total de Estudiante:</strong> 1</p>
<p><strong>Cantidad de Aprobados:</strong> 0</p>
<p><strong>Cantidad de Reprobados:</strong> 1</p>
<p><strong>Porcentaje de Reprobacion:</strong> 100</p>
<hr>
</body>
</html>

```

Reporte: rendimiento.html

```

<body>
<p>Nombre del Estudiante: Juan</p>
<p>Carnet: 202012345</p>
<p>Carrera: civil</p>
<p>Semestre: 5</p>
<p>Promedio General: 88.75</p>
<p>Cursos Aprobados: 2</p>
<p>Cursos Reprobados: 0</p>
<p><u>Creditos Acumulados</u>: 10</p>
<hr>
<body>
<p>Nombre del Estudiante: Maria</p>
<p>Carnet: 202013456</p>
<p>Carrera: industrial</p>
<p>Semestre: 3</p>
<p>Promedio General: 78</p>
<p>Cursos Aprobados: 1</p>
<p>Cursos Reprobados: 0</p>
<p><u>Creditos Acumulados</u>: 5</p>
<hr>
</body>

```

Reporte: ReportesAnálisisCarrera.html

```
<head>
<title>Análisis Carrera</title>
</head>
<h2>Carrera: Electronica</h2>
<p>Total estudiantes: 0</p>
<p>Promedio general: 0</p>
<p>Cursos disponibles: 0</p>
<p>Estudiantes por semestre:</p>
<ul>
</ul>
<hr>
<h2>Carrera: Sistemas</h2>
<p>Total estudiantes: 2</p>
<p>Promedio general: 50</p>
<p>Cursos disponibles: 4</p>
<p>Estudiantes por semestre:</p>
<ul>
<li>Semestre 1: 1 estudiantes</li>
<li>Semestre 5: 1 estudiantes</li>
</ul>
```

CONCLUSIONES FINALES

Logros Alcanzados

Lectura exitosa de archivos .lfp con validación en dos niveles

Separación clara entre análisis léxico y semántico

Generación automática de 5 reportes HTML funcionales

Manejo de errores que permite identificar registros inválidos

Estructura modular que facilita mantenimiento y mejoras

Aprendizajes Obtenidos

Comprensión profunda de las fases de un compilador (léxica, sintáctica, semántica)

Manejo avanzado de archivos en C++ con ifstream

Validación de datos en múltiples niveles

Generación de reportes en formato HTML

Trabajo con estructuras y vectores en C++

Importancia de la organización del código en archivos separados

Limitaciones Encontradas

Limitación	Descripción
Ordenamiento burbuja	Ineficiente para muchos datos, pero funcional para el alcance
Rutas fijas	No permite al usuario elegir ubicación de archivos

Recomendaciones para Mejoras Futuras

Implementar ordenamiento con sort() de algorithm para mejor rendimiento

Agregar verificación de integridad referencial (que cursos y estudiantes existan)

Permitir al usuario especificar rutas de archivos personalizadas

Agregar gráficos estadísticos simples en los reportes

Implementar manejo de excepciones con try-catch

Conclusión General

El sistema cumple satisfactoriamente con todos los requerimientos de la práctica: carga de archivos con validación léxica y semántica, procesamiento de datos, cálculos estadísticos y generación de 5 reportes en HTML. A pesar de las limitaciones propias del alcance del proyecto, la base está sólida y permite entender los conceptos fundamentales de análisis de lenguajes y procesamiento de datos.